

实验二：数值线性代数

实验性质

本实验为验证性实验。

实验目标

- 掌握数值线性代数的一般算法：LU 分解、SOR
- 学会导入并使用数值计算库：numpy、scipy、matplotlib
- 生成实验用的数据和函数
- 学会使用线性代数建模并求解问题

先决条件

具有 Python 编程的基本知识，了解与线性代数有关的数学概念

实验材料

- 一台带有 Python 解释器的计算机
- 可以运行 Jupyter Lab 的实验环境

实验过程说明

使用数值计算库

以下介绍的库不仅在数值计算领域，在数据分析、机器学习、科学研究等领域也有着广泛的应用。

1. NumPy

NumPy (Numerical Python) 是Python的一个开源库，用于支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库。NumPy是科学计算中常用的一个库，特别是对于那些涉及到大量线性代数运算的。

例：创建一个NumPy数组并进行简单运算

```
1 import numpy as np
2
3 # 创建一个2x2的数组
4 a = np.array([[1, 2], [3, 4]])
5
6 # 计算所有元素的和
7 print(np.sum(a)) # 输出 "10"
8
9 # 计算每一列的和
10 print(np.sum(a, axis=0)) # 输出 "[4 6]"
```

`np.dot` 是NumPy库中的一个函数，用于计算两个数组的点积（内积）或矩阵乘法。

`np.linalg` 是NumPy库中的线性代数模块，提供了一系列用于线性代数运算的函数。该模块包含了许多常用的线性代数操作，例如计算矩阵的逆、求解线性方程组、计算特征值和特征向量等。一些 `np.linalg` 模块中常用函数的说明如下：

- `np.linalg.inv()`：计算矩阵的逆矩阵
- `np.linalg.det()`：计算矩阵的行列式
- `np.linalg.eig()`：计算方阵的特征值和特征向量
- `np.linalg.solve()`：解线性方程组 $Ax = b$ ，其中 A 为系数矩阵， b 为常数向量
- `np.linalg.norm()`：计算向量或矩阵的范数
- `np.linalg.svd()`：奇异值分解，将矩阵分解为三个矩阵的乘积

2. SciPy

SciPy (Scientific Python) 是建立在NumPy之上的一款方便、易于使用、专为科学和工程设计的Python工具包。它为线性代数、数值积分和统计学等提供了许多实用的科学计算功能。

例：使用SciPy解线性方程组

```
1  from scipy.linalg import solve
2
3  # 定义方程组的系数矩阵和常数项
4  a = np.array([[3,1], [1,2]])
5  b = np.array([9,8])
6
7  # 解线性方程组Ax = b
8  x = solve(a, b)
9
10 print(x) # 输出解向量
```

3. Matplotlib

Matplotlib是Python的2D绘图库，可以生成出版质量级别和交互环境下的图形。通过Matplotlib，开发者仅需要几行代码，便可以生成直方图、功率谱、条形图、误差图、散点图等。

例：绘制简单的折线图

```
1  import matplotlib.pyplot as plt
2
3  # 准备数据
4  x = np.linspace(0, 10, 100)
5  y = np.sin(x)
6
7  # 绘制折线图
8  plt.plot(x, y)
9
10 # 显示图形
11 plt.show()
```

数值线性代数实验

生成实验数据

下面的示例代码可以从你的学号生成用来练习用的矩阵。

```
1 import numpy as np
2
3 def generate_matrix(school_number):
4     class_code = int(str(school_number)[-4:-2])
5     individual_code = int(str(school_number)[-2:])
6     n = class_code
7     A = np.zeros((n, n))
8     for i in range(n):
9         for j in range(n):
10             if i == j:
11                 A[i, j] = individual_code
12             else:
13                 A[i, j] = (individual_code % (j+1)) / (n+1)
14     b = np.array([(i + 1 + individual_code) * 2 for i in range(n)])
15
16     return A, b
```

测试用例

```
1 school_number = 22030501 # 换成你的学号
2 A, b = generate_matrix(school_number)
3 print("A:\n", A)
4 print("b:\n", b)
```

解线性方程组的直接法

直接方法通过一系列精确的算术运算找到方程组的精确解。其中，Gauss消元法和LU分解是最常用的两种直接方法。

Gauss消元法

Gauss消元法是一种通过行操作将线性方程组的系数矩阵转换为上三角形式（或行最简形式），从而便于求解的方法。分为两个主要步骤：前向消元和回代。

- **前向消元**：通过一系列的行加减操作，消除下三角的系数，使矩阵变为上三角形式。具体来说，从第一行开始，用当前行消除下方各行中对应列的系数。重复此过程，直到所有下方的系数都被消除。
- **回代**：一旦获得上三角矩阵，就从最后一行开始逐行向上求解未知数。因为矩阵是上三角的，所以每一行只有一个新的未知数，可以直接求解。

以下是示例代码

```
1 def gauss_elimination(A, b):
2     """
3     使用Gauss消元法求解线性方程组Ax=b
4
5     参数：
6     A -- 系数矩阵
```

```

7      b -- 常数项向量
8
9      返回：
10     线性方程组的解向量x，如果存在唯一解；
11     否则返回None。
12     """
13     n = len(A)
14     Ab = np.hstack((A.astype(float), b.reshape(-1, 1).astype(float)))
15
16     # 前向消元
17     for i in range(n-1):
18         if Ab[i, i] == 0:
19             print("主元为零，无法继续消元")
20             return None
21         for j in range(i+1, n):
22             factor = Ab[j, i] / Ab[i, i]
23             Ab[j, i:] -= factor * Ab[i, i:]
24
25     # 检查是否存在唯一解
26     if Ab[n-1, n-1] == 0:
27         print("没有唯一解")
28         return None
29
30     # 回代
31     x = np.zeros(n)
32     for i in range(n-1, -1, -1):
33         x[i] = (Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])) / Ab[i, i]
34
35     return x
36
37     # 用例
38     x = gauss_elimination(A, b)
39     if x is not None:
40         print("唯一解:", x)

```

LU分解

LU分解是将矩阵分解为一个下三角矩阵（L）和一个上三角矩阵（U）的乘积。主要优点是一旦完成分解，就可以复用于求解多个不同的常数向量 b 的线性方程组，而无需重新执行消元过程。

- **L矩阵**：下三角矩阵，其中所有对角线上的元素通常设为1。
- **U矩阵**：上三角矩阵，其元素由原矩阵通过Gauss消元法变换而来。

LU分解的步骤类似于Gauss消元，不同之处在于LU分解会同时保存L和U两个矩阵。对于方程 $Ax = b$ ，一旦有了L和U，可以首先解 $Ly = b$ （前向替换），然后解 $Ux = y$ （回代）来找到解向量 x 。

实验要求

根据课件提供的算法伪代码，参考上述Gauss消元实现，使用Python语言编写一个函数 `doolittle` 对输入的矩阵，即根据学号生成的 A 进行LU分解。如名字所示，请使用Doolittle形式的分解。

解线性方程组的迭代法

迭代方法通过从一个初始估计开始，逐步逼近方程组的精确解。与直接方法相比，迭代方法通常在处理大型稀疏矩阵时更为有效。Jacobi法、Gauss-Seidel法和Successive Over-Relaxation（SOR）法是最常用的迭代方法。

Jacobi法

Jacobi迭代法是最简单的线性方程组迭代解法之一。它将每个未知数的新估计值基于上一次迭代中其他未知数的值来计算。对于方程组 $Ax = b$ ，其中 A 是系数矩阵， b 是常数项向量，迭代公式为：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

这里， $x^{(k+1)}$ 是下一次迭代的解向量， $x^{(k)}$ 是当前迭代的解向量。

以下是示例代码

```
1  def jacobi_iteration(A, b, p, tau, N):
2      """
3      使用Jacobi迭代法求解线性方程组Ax=b
4
5      参数：
6      A -- 方阵，系数矩阵
7      b -- 向量，常数项
8      p -- 向量，初始估计解
9      tau -- 容差，用于判断迭代是否收敛
10     N -- 最大迭代次数
11
12     返回：
13     近似解向量x，如果迭代成功；
14     或超出迭代次数的信息
15     """
16     n = A.shape[0]
17     x = np.zeros_like(p)
18
19     for k in range(N):
20         for i in range(n):
21             sum_j = np.dot(A[i, :], p) - A[i, i] * p[i]
22             x[i] = (b[i] - sum_j) / A[i, i]
23             if np.linalg.norm(x - p, np.inf) < tau:
24                 return x
25         p[:] = x
26
27     print("达到最大迭代次数，失败")
28     return None
29
30 # 用例
31 x = jacobi_iteration(A, b, np.zeros(b.shape), 1e-5, 100)
32 if x is not None:
33     print("近似解:", x)
```

Gauss-Seidel法和SOR法

Gauss-Seidel迭代法是Jacobi法的一个改进，它在计算当前未知数的新估计值时，会立即使用已经计算出的新值，而不是上一次迭代的值。这种方法通常能加速收敛。Gauss-Seidel法的迭代公式为：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

SOR法是Gauss-Seidel法的一种变体，通过引入一个松弛因子 ω 来调整每次迭代的步长，以期获得更快的收敛速度。当 $\omega = 1$ 时，SOR法退化为Gauss-Seidel法。SOR法的迭代公式为：

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

其中， ω 是一个大于0的实数，选择合适的 ω 值对于加速收敛非常关键。

实验要求

根据课件提供的算法伪代码，参考上述Jacobi迭代的实现，使用Python语言编写一个函数 `sor` 对根据学号生成的 A 和 b 进行求解。

信号处理

加权最小二乘法（Weighted Least Squares, WLS）是一种常用的数据处理方法，用于信息去噪或数据拟合问题。它在最小二乘法的基础上引入了权重因子，可以对不同数据点进行加权处理，以降低噪声的影响或调整不同数据点的重要性。

$$\min_x \frac{1}{2} \|x - b\|_2^2 + \frac{1}{2} \alpha \|Dx\|_2^2$$

其中， D 是离散的二阶微分算子，注意 `np.diag` 可以生成一个对角矩阵

```
1 | D = np.diag(-2*e) + np.diag(e[1:], 1) + np.diag(e[1:], -1)
```

$$\frac{1}{2} (x - b)^T (x - b) + \frac{1}{2} \alpha (Dx)^T (Dx)$$

展开后得到：

$$\frac{1}{2} (x^T x - 2x^T b + b^T b) + \frac{1}{2} \alpha (x^T D^T D x)$$

接下来我们要找到最小值点，对 x 求导，并且令导数等于 0

重新整理得到：

$$(I + \alpha D^T D)x = b$$

这就是正规方程的形式。通过求解这个方程，我们可以找到目标函数的最小值点。

实验要求

分别给出权重因子 $\alpha = 0.1, 0.5, 1, 5, 10, 50$ 时候的去噪结果，并使用 `matplotlib` 画出相应的图像

撰写实验报告

1. 总结该实验，包括实验目标、实验方法和实验过程。
2. 描述实验实施的具体情况和结果，包括遇到的问题和挑战，已经如何解决问题的思路。
3. 分析实验结果，并根据所使用的数据和方法得出结论。
4. 将实验报告誊写到报告书模版文档。